

1-

```
console.log(0.1 * 3 === 0.3);  
console.log(9007199254740993);
```

R:

console.log(0.1 * 3 === 0.3); -> Saída foi false. O problema encontrado foi na execução, já que no nodejs(e no java script em si) o valor de 0.1 não é exatamente 0,1 e nem a multiplicação do mesmo resulta no resultado exato 0.3

console.log(9007199254740993); -> O Node retornou 9007199254740992, um a menos que o “esperado”. O erro encontrado é na execução, já que este número não existe pela contagem de bites, logo, ele estoura e retorna com um a menos.

2-

```
palavra = "maçã"  
print(len(palavra), len(palavra.encode())) # encode() usa UTF-8 por  
padrão
```

R:

Saída obtida: 4 6, o problema encontrado é na execução que, quando transformamos ç e ã em encode do UTF-8, eles viram dois caracteres diferentes, por passarem da quantidade de 16 bits permitida.

3-

```
func main() {  
    var b byte = 255  
    b++  
    fmt.Println(b)  
}
```

R:

Saída obtida: 0. O “erro” estaria ligado ao fato que um byte tem no máximo 255 de tamanho, e, ao incrementá-lo, o mesmo retorna a 0.

4-

```
public class Main {  
    public static void main(String[] args) {  
        int[] v = new int[3];  
        System.out.println(v[0]);  
        System.out.println(v[3]);  
    }  
}
```

R: A saída é: "0" e então um erro é detectado na **execução** porque o programa tenta acessar um elemento fora do alcance do vetor instanciado assim que roda a linha **System.out.println(v[3]);**

5-

```
fn main() {  
    let s = String::from("oi");  
    let t = s;  
    println!("{}", s, t);  
}
```

R:

Não há saída, na hora de executar o programa, apresenta erro. Em Rust, ao fazer **let t = s;** não é feito a ação de “copiar” o conteúdo, e sim, ele substitui a variável, então a variável **s** foi atribuída a variável **t**.

```
int main(void) {  
    union { int i; float f; } u;  
    u.f = 1.0f;  
    printf("%d\n", u.i);  
    return 0;  
}
```

R:

Saída: 1065353216

O problema não é detectado nunca.